Introduction

The increasing reliance on digital platforms for entertainment services has made online movie ticket booking systems a critical component of modern cinema operations. These systems require efficient handling of diverse and interconnected data, including user profiles, movie catalogs, theater configurations, and booking transactions. Ensuring accuracy, scalability, and fast data access is essential for delivering a seamless user experience.

This project presents a professionally designed **Online Movie Ticket Booking System using MongoDB**, a powerful document-oriented NoSQL database. The system is structured using well-defined **Users, Movies, Theaters, and Bookings collections** and demonstrates comprehensive use of MongoDB features such as **CRUD operations, query operators, aggregation pipelines, and indexing techniques**. The project not only reflects real-world database design practices but also serves as a strong academic and practical reference for database management, system implementation, and examination preparation.

Abstract

This project presents an **Online Movie Ticket Booking System** developed using **MongoDB**, designed to manage users, movies, theaters, and booking transactions in an efficient and structured manner. The system models real-world relationships through well-defined collections, ensuring flexibility and scalability in data handling.

The project demonstrates essential MongoDB functionalities, including **CRUD operations**, **query operators**, **aggregation pipelines**, and **indexing**. These features enable accurate data retrieval, efficient updates, and advanced data analysis such as booking statistics, user preferences, and theater performance.

Overall, the project highlights MongoDB's effectiveness as a modern NoSQL database for building scalable applications. It serves as a practical and academic reference for understanding database design, query optimization, and data analytics, making it suitable for final-year projects, practical examinations, and interview preparation.

MOVIE TICKET BOOKING SYSTEM

1. Database Design

Collections Used:

1. users
2. movies
3. theaters
4. Bookings Details

2.Sample Document Structure

Users Collection:

{"user_id": 1, "full_Name": "Luciano Brezlaw", "phone_Number": "273-273-6966", "password": "vD3$qckD}lqFS_#" }

{“user_id": 2, "full_Name": "Kalle Corington", "phone_Number": "365-661-4079", "password": "nS5@$lbh&X!l?7(" }

Movies Collection:

{"movieId": 101,"movieName": "Eko","casts": ["Vijay", "Trisha", "Sanjay Dutt"],"rating": 4.4,"genre": "Action"},

{"movieId": 104,"movieName": "Kalki 2898 AD","casts": ["Prabhas", "Deepika Padukone", "Amitabh Bachchan"],"rating": 4.6,"genre": "Sci-Fi"},

Theater Collection:

{ "theaterId": 201, "theaterName": "PVR Lulu Mall", "location": { "district": "Ernakulam", "city": "Kochi" }, "screen": [{ "name": "Screen 1", "seats": 150, "amount": 180 }, { "name": "Screen 2", "seats": 120, "amount": 160 }], "rating": 4.5 }

{ "theaterId": 202, "theaterName": "INOX Marina Mall", "location": { "district": "Chennai", "city": "Chennai" }, "screen": [{ "name": "Screen A", "seats": 200, "amount": 220 }, { "name": "Screen B", "seats": 180, "amount": 200 }], "rating": 4.6 }

Booking Collection:

{ " bookingId": 301, "userId": 1, "movieId": 101, "theaterId": 201, "screen": "Screen 1", "payment": "UPI" },

{ bookingId": 302, "userId": 2, "movieId": 102, "theaterId": 202, "screen": "Screen A", "payment": "CARD" }

Operations and Answers

3. CRUD Operations

3.1 Insert Operations

a) Insert a new user with user_id 11 and valid details

```
theatre_Project> db.Users.insertOne([
                    {"user_id": 11,
                     "full_Name": "Luciano Brezlaw",
                     "phone_Number": "273-273-6966",
                     "password": "vD3$qckD}lqFS_#" }
                ])
```

b) Insert a new movie with genre **Thriller** and rating **4.3**.

```
theatre_Project> db.Movies.insertOne([
                    {"movieId": 111,
                     "movieName": "pravinkoodu shappu",
                     "casts":["basil joseph", "saubin shair"],
                     "rating": 4.3,
                     "genre": "Triller"}
                ])
```

3.2 Read Operations

a) Find user details using **user_id = 5**

```
> db.Users.findOne({
    user_id:5
  })
< {
    _id: ObjectId('69608dcdcbe2d629d96832f7'),
    user_id: 5,
    full_Name: 'Georg Coney',
    phone_Number: '517-223-7693',
    password: 'rI1}d6(rS5BV'
  }
theatre_Project>
```

b) Get movies belonging to **Sci-Fi** genre

```
> db.Movies.find(
    { genre: "Sci-Fi" }
  )
< {
    _id: ObjectId('696094e563ee47234e27a9f6'),
    movieId: 104,
    movieName: 'Kalki 2898 AD',
    casts: [
      'Prabhas',
      'Deepika Padukone',
      'Amitabh Bachchan',
      'Kamal Haasan'
    ],
    rating: 4.6,
    genre: 'Sci-Fi'
  }
  {
    _id: ObjectId('696094e563ee47234e27a9f9'),
    movieId: 109,
    movieName: 'Avatar 3',
    casts: [
      'Sam Worthington',
      'Zoe Saldana',
      'Sigourney Weaver'
    ],
    rating: 4.7,
    genre: 'Sci-Fi'
  }
```

c) Find all bookings done using **UPI** payment

```
> db.Booking_Detials.find(
    { payment: "UPI" }
  )
< {
    _id: ObjectId('696095ee63ee47234e27aa08'),
    bookingId: 301,
    userId: 1,
    movieId: 101,
    theaterId: 201,
    screen: 'Screen 1',
    payment: 'UPI'
  }
  {
    _id: ObjectId('696095ee63ee47234e27aa0a'),
    bookingId: 303,
    userId: 3,
    movieId: 103,
    theaterId: 203,
    screen: 'Screen X',
    payment: 'UPI'
  }
```

3.3 Update Operations

a) Update phone number of user with **user_id = 3**

```
theatre_Project> db.Users.updateOne(
                   { user_id: 3 },
                   { $set: { phone_Number: "999-888-7777" } }
                 )
```

b) Increase movie rating of **"Salaar"** to **4.6**

```
theatre_Project> db.movies.updateOne(
                   { movieName: "Salaar: Part 1 - Ceasefire" },
                   { $set: { rating: 4.6 } }
                 )
```

c) Update seat price of **Screen 1** in **PVR Forum Mall**.

```
theatre_Project> db.theater.updateOne(
                  {theaterName: "PVR Forum Mall",
                    "screen.name": "Screen 1"},
                  { $set: { "screen.$.amount": 210 }}
                )
```

3.4 Delete Operations

a) Delete a user with **user_id = 11**.

```
theatre_Project> db.users.deleteOne(
                    { user_id: 11 }
                  )
```

4. Query Operators

4.1 Comparison Operators ($eq, $ne, $gt, $gte, $lt, $lte, $in, $nin)

a) Find movies with rating > 4.6.

```
theatre_Project> db.Movies.find(
                  { rating: { $gt: 4.6 } }
                )
```

```
< {
    _id: ObjectId('696094e563ee47234e27a9f9'),
    movieId: 109,
    movieName: 'Avatar 3',
    casts: [
      'Sam Worthington',
      'Zoe Saldana',
      'Sigourney Weaver'
    ],
    rating: 4.7,
    genre: 'Sci-Fi'
  }
  {
    _id: ObjectId('696094e563ee47234e27a9fa'),
    movieId: 110,
    movieName: 'Dune: Part Two',
    casts: [
      'Timothée Chalamet',
      'Zendaya',
      'Rebecca Ferguson'
    ],
    rating: 4.8,
    genre: 'Sci-Fi'
  }
```

b) Find theaters where screen seat count is ≥ 200

```
theatre_Project> db.Theaters.find(
                    {"screen.seats": { $gte: 200 } }
                )
```

```
  {
    _id: ObjectId('6960957363ee47234e27aa02'),
    theaterId: 205,
    theaterName: 'INOX Megaplex',
    location: {
      district: 'Mumbai Suburban',
      city: 'Mumbai'
    },
    screen: [
      {
        name: 'Screen 5',
        seats: 220,
        amount: 250
      }
    ],
    rating: 4.7
  }
  {
    _id: ObjectId('6960957363ee47234e27aa03'),
    theaterId: 206,
    theaterName: 'Sathyam Cinemas',
    location: {
      district: 'Chennai',
      city: 'Chennai'
    },
    screen: [
```

c) Find users whose user_id is between 3 and 8

```
theatre_Project> db.Users.find(
                  { user_id: { $gte: 3, $lte: 8 } }
                )
```

```
< {
    _id: ObjectId('69608dcdcbe2d629d96832f5'),
    user_id: 3,
    full_Name: 'Jefferson Sante',
    phone_Number: '999-888-7777',
    password: 'bS3&vLH*wu&'
  }
  {
    _id: ObjectId('69608dcdcbe2d629d96832f6'),
    user_id: 4,
    full_Name: 'Kalvin Godfroy',
    phone_Number: '162-990-8018',
    password: 'fE1">xI&9'
  }
  {
    _id: ObjectId('69608dcdcbe2d629d96832f7'),
    user_id: 5,
    full_Name: 'Georg Coney',
    phone_Number: '517-223-7693',
    password: 'rI1}d6(rS5BV'
  }
```

4.2 Logical Operators ($and, $or, $not, $nor)

a) Find movies that are **Action genre AND rating ≥ 4.5**

```
theatre_Project> db.Movies.find(
                    {genre: "Action",
                      rating: { $gte: 4.5 }}
                  )
```

```
{
  _id: ObjectId('696094e563ee47234e27a9f8'),
  movieId: 108,
  movieName: 'Pushpa 2: The Rule',
  casts: [
    'Allu Arjun',
    'Rashmika Mandanna',
    'Fahadh Faasil'
  ],
  rating: 4.6,
  genre: 'Action'
}
```

b) Find theaters located in **Chennai OR Kochi**

```
theatre_Project> db.Theaters.find(
                  {$or: [
                      { "location.city": "Chennai" },
                      { "location.city": "Kochi" }
                    ]})
```

```
{
  _id: ObjectId('6960957363ee47234e27a9fe'),
  theaterId: 201,
  theaterName: 'PVR Lulu Mall',
  location: {
    district: 'Ernakulam',
    city: 'Kochi'
  },
  screen: [
    {
      name: 'Screen 1',
      seats: 150,
      amount: 180
    },
    {
      name: 'Screen 2',
      seats: 120,
      amount: 160
    }
  ],
  rating: 4.5
}
```

d) Find bookings where payment is **UPI** or **CARD**

```
theatre_Project> db.Booking_Detials.find(
                  {payment: { $in: ["UPI", "CARD"] }}
                )
```

```
< {
    _id: ObjectId('696095ee63ee47234e27aa08'),
    bookingId: 301,
    userId: 1,
    movieId: 101,
    theaterId: 201,
    screen: 'Screen 1',
    payment: 'UPI'
  }
  {
    _id: ObjectId('696095ee63ee47234e27aa09'),
    bookingId: 302,
    userId: 2,
    movieId: 102,
    theaterId: 202,
    screen: 'Screen A',
    payment: 'CARD'
  }
  {
    _id: ObjectId('696095ee63ee47234e27aa0a'),
    bookingId: 303,
    userId: 3,
    movieId: 103,
```

4.3 Logical Operators ($exists $type)

a) Find theaters where **screen.amount exists**

```
theatre_Project> db.Theaters.find(
                    { "screen.amount": { $exists: true } }
                  )
```

```
{
  _id: ObjectId('6960957363ee47234e27a9fe'),
  theaterId: 201,
  theaterName: 'PVR Lulu Mall',
  location: {
    district: 'Ernakulam',
    city: 'Kochi'
  },
  screen: [
    {
      name: 'Screen 1',
      seats: 150,
      amount: 180
    },
    {
      name: 'Screen 2',
      seats: 120,
      amount: 160
    }
  ],
  rating: 4.5
}
```

4.4 Array Operators ($all, $size, $elemmatch)

a) Find movies with **more than 3 cast members**

```
theatre_Project> db.Movies.find(
                    { casts: { $size: 3 } }
                )
```

```
< {
    _id: ObjectId('696094e563ee47234e27a9f5'),
    movieId: 101,
    movieName: 'Eko',
    casts: [
      'Vijay',
      'Trisha',
      'Sanjay Dutt'
    ],
    rating: 4.4,
    genre: 'Action'
  }
  {
    _id: ObjectId('696094e563ee47234e27a9f7'),
    movieId: 106,
    movieName: 'Salaar: Part 1 - Ceasefire',
    casts: [
      'Prabhas',
      'Prithviraj Sukumaran',
      'Shruti Haasan'
    ],
    rating: 4.4,
    genre: 'Action'
  }
```

b) Find movies where the **casts array contains BOTH "Prabhas" and "Deepika Padukone"**

```
theatre_Project> db.Movies.find({
                    casts:
                    { $all: ["Prabhas", "Deepika Padukone"] }
                  })
```

```
{
  _id: ObjectId('696094e563ee47234e27a9f6'),
  movieId: 104,
  movieName: 'Kalki 2898 AD',
  casts: [
    'Prabhas',
    'Deepika Padukone',
    'Amitabh Bachchan',
    'Kamal Haasan'
  ],
  rating: 4.6,
  genre: 'Sci-Fi'
}
```

5. Aggregation Operations ($match, $group, $project, $sort, $limit, $skip, $lookup, $unwind, $count)

I. Basic Aggregation Operations

a) Count total number of users

```
theatre_Project> db.Users.countDocuments()
```

b) Count total number of movies

```
theatre_Project> db.Movies.countDocuments()
```

II. $group

a) Count total bookings per payment method

```
theatre_Project> db.Booking_Detials.aggregate([
                    { $group: {
                        _id: "$payment",
                        totalBookings: { $sum: 1 } }
                    }])
```

```
< {
    _id: 'COD',
    totalBookings: 1
  }
  {
    _id: 'UPI',
    totalBookings: 5
  }
  {
    _id: 'CARD',
    totalBookings: 4
  }
```

b) Find average movie rating by genre

```
theatre_Project> db.Movies.aggregate([
                  {$group: {
                      _id: "$genre",
                      avgRating: { $avg: "$rating" }}
                  }])
```

```
< {
    _id: null,
    avgRating: null
  }
  {
    _id: 'Drama',
    avgRating: 4.800000000000001
  }
  {
    _id: 'Action',
    avgRating: 4.4
  }
  {
    _id: 'Sci-Fi',
    avgRating: 4.7
  }
```

c) Find total bookings per movieId

```
theatre_Project> db.Booking_Detials.aggregate([
                    {$group: {_id: "$movieId",
                        totalBookings: { $sum: 1 } }
                    }])
```

```
< {
    _id: 110,
    totalBookings: 1
  }
  {
    _id: 106,
    totalBookings: 1
  }
  {
    _id: 102,
    totalBookings: 1
  }
  {
    _id: 101,
    totalBookings: 1
  }
  {
    _id: 103,
    totalBookings: 1
  }
```

d) Find average movie rating by genre

```
theatre_Project> db.Movies.aggregate([
                    { $group: {
                        _id: "$genre",
                        avgRating: { $avg: "$rating" } }
                    }])
```

```
[
  {
    _id: null,
    avgRating: null
  }
  {
    _id: 'Sci-Fi',
    avgRating: 4.7
  }
  {
    _id: 'Action',
    avgRating: 4.4
  }
  {
    _id: 'Drama',
    avgRating: 4.800000000000001
  }
```

III. $ lookup

a) Get booking details with user name, movie name, and theater name

```
theatre_Project> db.Booking_Detials.aggregate([
                    { $lookup: { from: "users",
                                 localField: "userId",
                                 foreignField: "user_id",
                                 as: "user" } },
                    { $lookup: { from: "movies",
                                 localField: "movieId",
                                 foreignField: "movieId",
                                 as: "movie" } },
                    { $lookup: { from: "theaters",
                                 localField: "theaterId",
                                 foreignField: "theaterId",
                                 as: "theater" } }
                ])
```

```
< {
    _id: ObjectId('696095ee63ee47234e27aa08'),
    bookingId: 301,
    userId: 1,
    movieId: 101,
    theaterId: 201,
    screen: 'Screen 1',
    payment: 'UPI',
    user: [],
    movie: [],
    theater: []
  }
  {
    _id: ObjectId('696095ee63ee47234e27aa09'),
    bookingId: 302,
    userId: 2,
    movieId: 102,
    theaterId: 202,
    screen: 'Screen A',
    payment: 'CARD',
    user: [],
    movie: [],
    theater: []
  }
  {
    _id: ObjectId('696095ee63ee47234e27aa0a'),
    bookingId: 303,
```

b) Display all bookings along with city and screen name

```
theatre_Project> db.Booking_Detials.aggregate([
                  { $lookup: {
                      from: "theaters",
                      localField: "theaterId",
                      foreignField: "theaterId",
                      as: "theater" }
                },
                  { $project: {
                      bookingId: 1,
                      screen: 1,
                      city: { $arrayElemAt: ["$theater.location.city", 0] } }
                  }])
```

```
< {
    _id: ObjectId('696095ee63ee47234e27aa08'),
    bookingId: 301,
    screen: 'Screen 1'
  }
  {
    _id: ObjectId('696095ee63ee47234e27aa09'),
    bookingId: 302,
    screen: 'Screen A'
  }
  {
    _id: ObjectId('696095ee63ee47234e27aa0a'),
    bookingId: 303,
    screen: 'Screen X'
  }
  {
    _id: ObjectId('696095ee63ee47234e27aa0b'),
    bookingId: 304,
    screen: 'Screen 1'
  }
  {
    _id: ObjectId('696095ee63ee47234e27aa0c'),
    bookingId: 305,
    screen: 'Screen 5'
  }
  {
    _id: ObjectId('696095ee63ee47234e27aa0d'),
    bookingId: 306,
```

IV. $match + $group

a) Find total bookings for Action movies

```
theatre_Project> db.Booking_Detials.aggregate([
                  { $match: { payment: "UPI" } },
                  {$lookup: {
                      from: "theaters",
                      localField: "theaterId",
                      foreignField: "theaterId",
                      as: "theater" } },
                  {$group: {
                      _id: { $arrayElemAt: ["$theater.location.city", 0] },
                      totalUPIBookings: { $sum: 1 } }}
                ])
```

```
< {
    _id: null,
    totalUPIBookings: 5
  }
```

V. $sort / $limit

a) Find top 3 highest-rated movies

```
theatre_Project> db.Movies.find().sort({ rating: -1 }).limit(3)
```

```
< {
    _id: ObjectId('696094e563ee47234e27a9fb'),
    movieId: 111,
    movieName: 'Oppenheimer',
    casts: [
      'Cillian Murphy',
      'Emily Blunt',
      'Robert Downey Jr.'
    ],
    rating: 4.9,
    genre: 'Drama'
  }
  {
    _id: ObjectId('696094e563ee47234e27a9fa'),
    movieId: 110,
    movieName: 'Dune: Part Two',
    casts: [
      'Timothée Chalamet',
      'Zendaya',
      'Rebecca Ferguson'
    ],
    rating: 4.8,
    genre: 'Sci-Fi'
  }
```

VI. $project

a) Display movie name and rating only

```
theatre_Project> db.Movies.find(
                  {},{ _id: 0, movieName: 1, rating: 1 }
                )
```

```
< {
    movieName: 'Eko',
    rating: 4.4
  }
  {
    movieName: 'Kalki 2898 AD',
    rating: 4.6
  }
  {
    movieName: 'Salaar: Part 1 – Ceasefire',
    rating: 4.4
  }
  {
    movieName: 'Pushpa 2: The Rule',
    rating: 4.6
  }
```

VII. $unwind

a) Unwind movie casts and list each actor separately

```
theatre_Project> db.Movies.aggregate([
                  { $unwind: "$casts" },
                  {$project: {
                      movieName: 1,
                      actor: "$casts"}}
                ])
```

```
{
  _id: ObjectId('696094e563ee47234e27a9f5'),
  movieName: 'Eko',
  actor: 'Vijay'
}
{
  _id: ObjectId('696094e563ee47234e27a9f5'),
  movieName: 'Eko',
  actor: 'Trisha'
}
{
  _id: ObjectId('696094e563ee47234e27a9f5'),
  movieName: 'Eko',
  actor: 'Sanjay Dutt'
}
{
  _id: ObjectId('696094e563ee47234e27a9f6'),
  movieName: 'Kalki 2898 AD',
  actor: 'Prabhas'
}
```

Conclusion

This project successfully demonstrates the design and implementation of an Online Movie Ticket Booking System using MongoDB, highlighting its suitability for managing real-world, data-intensive applications. By organizing data into structured collections such as Users, Movies, Theaters, and Bookings, the system efficiently handles complex relationships while maintaining flexibility and scalability.

Through the use of CRUD operations, the project ensures complete data management capabilities, while query operators and aggregation pipelines enable powerful data retrieval and meaningful analysis such as booking trends, user preferences, and theater performance. The implementation of indexing techniques further enhances query performance and ensures data integrity.

Overall, this project provides a clear understanding of MongoDB's strengths as a modern NoSQL database and serves as a valuable academic and professional reference for database design, optimization, and application development.